At a high level, an inverted index is a data structure much like a dictionary, but with both the words and lists of documents the words are present in. This inverted index is the key to faster retrieval of documents during the full-text search phase. For each document that consists of full-text fields, the server creates a respective inverted index.

> **NOTE** Block k-dimensional (BKD) trees are special data structures used to hold non-text fields like numeric and geoshapes.

We've learned a bit of theory about inverted indexes, but now let's look at a simple example of how they work. Say we have two documents with one text field, `greeting`:

```
//Document 1
{
  "greeting":"Hello, WORLD"
}
//Document 2
{
  "greeting":"Hello, Mate"
}
```

In Elasticsearch, the analysis process is a complex function carried out by an analyzer module. The analyzer module is further composed of character filters, a tokenizer, and token filters. When the first document is indexed, as in the `greeting` field (a text field), an inverted index is created. Every full-text field is backed up by an inverted index. The value of the greeting "Hello, World" is analyzed, tokenized, and normalized into two words—*hello* and *world*—by the end of the process. But there are a few steps in between.

Let's look at the overall process (figure 3.18). The input line `<h2>Hello WORLD</h2>` is stripped of unwanted characters such as HTML markup. The cleaned-up data is split into tokens (most likely individual words) based on whitespace, thus forming

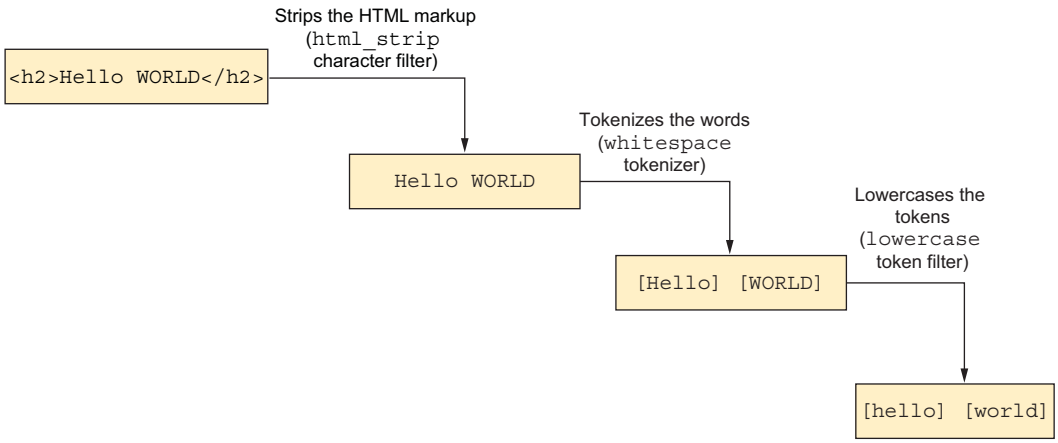


Figure 3.18 Text analysis procedure where Elasticsearch processes text